

Manual de seguimiento proyecto (Repositorio nombre asignado para la clase)

Descripción General

Este repositorio principal centraliza el desarrollo del proyecto colaborativo de la clase. **Todos los estudiantes deben trabajar en su repositorio personal** organizando sus actividades y tutorías, mientras que **uno de los integrantes** también gestionará el **repositorio principal**, asegurando la integración de las Historias de Usuario (HU) y la correcta organización del código compartido.

Estructura del Repositorio

Repositorio Principal (Compartido por el Grupo)

- **Ramas principales:**
 - **main** → Versión estable del proyecto.
 - **qa** → Pruebas de calidad antes de pasar a **main**.
 - **develop** → Desarrollo activo del proyecto. Aquí se fusionan las Historias de Usuario (HU).
- **Ramas de desarrollo:**
 - **feature/HU-{cod}** → Funcionalidades específicas (backend, frontend, database o documentación). Ejemplo: **feature/HU-01**.
 - **feature/activity** → Estructura para la evidencia del trabajo semanal (para el integrante encargado del repositorio principal).

Ejemplo de Organización en el Repositorio Principal

```
main/
qa/
develop/
feature/
|— HU-01/
|   |— backend/
|   |— frontend/
|— HU-02/
|   |— database/
|— HU-03/
|   |— api/
|— activity/
|   |— 01-week/
|       |— 01-session/
|       |— 02-session/
|   |— 02-week/
|       |— 01-session/
|       |— 02-session/
|   |— 05-week/
|       |— 01-session/
|       |— 02-session/
```

Repositorios Personales (Cada Estudiante)

Cada estudiante gestiona su trabajo en su propio repositorio personal con la siguiente estructura:

- **feature/activity** → Actividades semanales.
 - **01-week/01-session/** → Práctica sesión 1.
 - **01-week/02-session/** → Práctica sesión 2.
- **feature/tutoring** → Seguimiento de tutorías.
- **feature/project** → Desarrollo de HU propias. **Debe incluir un enlace al repositorio principal.**

Ejemplo de Organización en un Repositorio Personal

```
feature/
├── activity/
│   ├── 01-week/
│   │   ├── 01-session/
│   │   │   ├── ejercicio1.kt
│   │   │   └── ejercicio2.kt
│   │   └── 02-session/
│   └── 02-week/
│       └── 01-session/
├── tutoring/
│   ├── feedback-docs/
│   └── tutor-notes.md
└── project/
    ├── README.md (enlace al repositorio principal)
    ├── login-feature/
    │   ├── login.kt
    │   └── auth-service.kt
    └── user-profile/
        ├── profile-screen.kt
        └── avatar-upload.kt
```

Flujo de Trabajo

1. **Trabajo en repositorio personal:** Subir avances en **feature/activity**, **feature/tutoring** o **feature/project**.
2. **Sincronizar con el repositorio principal:** Mantener actualizado el código mediante **git fetch upstream** y **git merge upstream/develop**.
3. **Enviar cambios al proyecto principal:**
 - Crear un **Merge Request** desde **feature/HU-{cod}** a **develop** del repositorio principal.
 - Usar **Conventional Commits en inglés** en los mensajes de commit.
 - Seguir las reglas de nomenclatura y estructura del código.

Ejemplo de Convenciones de Commit (Conventional Commits)

```
feat(login): add user authentication module
fix(api): resolve bug in user data endpoint
refactor(database): optimize query performance
chore(ci): update GitHub Actions workflow
```

☑ Buenas Prácticas

- **Estructura organizada:** Mantener archivos ordenados y descriptivos.
- **Commits frecuentes y descriptivos:** Usar `feat:`, `fix:`, `docs:`, `refactor:`, etc.
- **No subir archivos innecesarios:** Configurar `.gitignore` correctamente.
- **Colaboración efectiva:** Revisar y probar código antes de fusionar.

Siguiendo estas reglas, garantizamos un proyecto bien estructurado y fácil de mantener. 🚀